

Put, get, remove, containsKey $\rightarrow O(1)$

get, set $O(1)$

$\rightarrow$ Array

① Key value pair

Name $\rightarrow$ marks
Key Value

$O(1)$ Raj-87

$O(1)$

Kum-85
Anil-77

Raj, 87	Kum, 85	Anil, 77
---------	---------	----------

$O(1)$

D:84	A:86	B:77	C:56
0	1	2	3

2k	1k	5k	6k
----	----	----	----

0 2k	1 1k	2 5k	3 6k
D:84	A:86	B:77	C:56

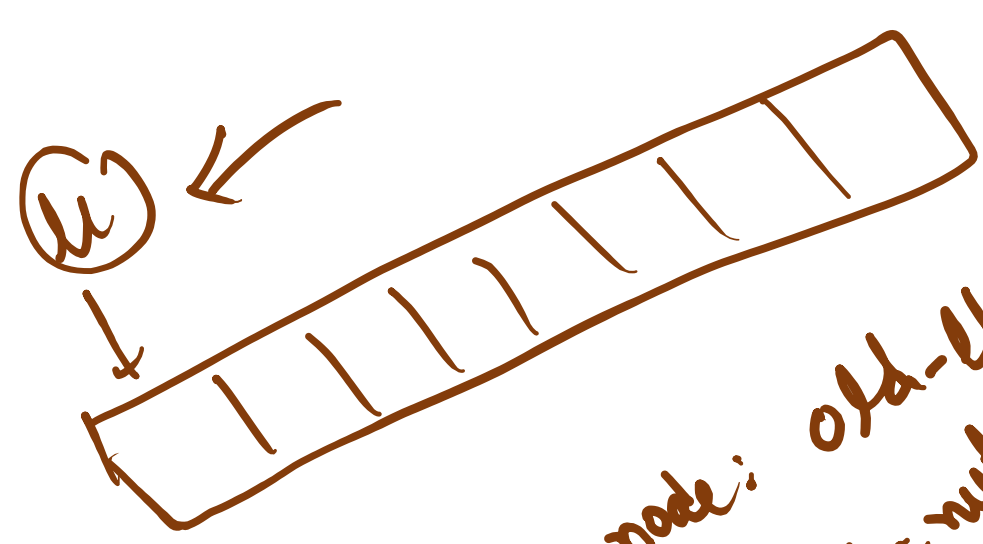
A $\rightarrow$ 86
B $\rightarrow$ 77
C $\rightarrow$ 56
D $\rightarrow$ 84
E $\rightarrow$ 72

A $5 \times 4 = 20$
B $6 \times 4 = 24$
C $7 \times 4 = 28$
D $8 \times 4 = 32$
E $9 \times 4 = 36$

HashCode
 $\rightarrow$ Hash fun

Polynomial

Chaining
LinkedList



Post node node: old-ll
while node != null
put C node key, node value
node = node.next



-D A B C
-A E F G
-E J K L

m=4-1

$O(n)$

① Load factor $\rightarrow LF$

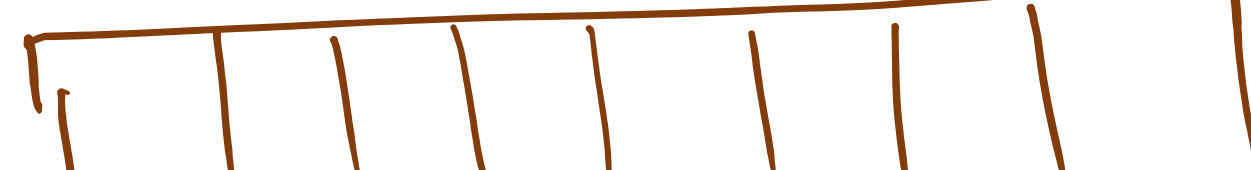
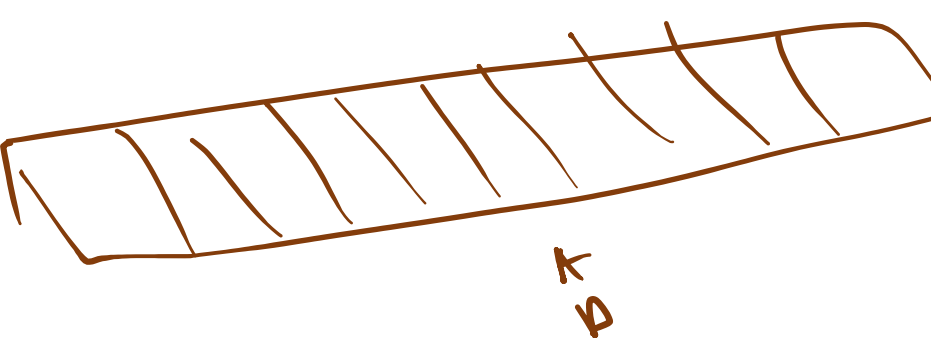
HF $\rightarrow 2.0$

total of element
bucket size $= \frac{12}{4} = 3.0$

$O(1)$

① re-hashing

$\frac{12}{8} = 1.5$



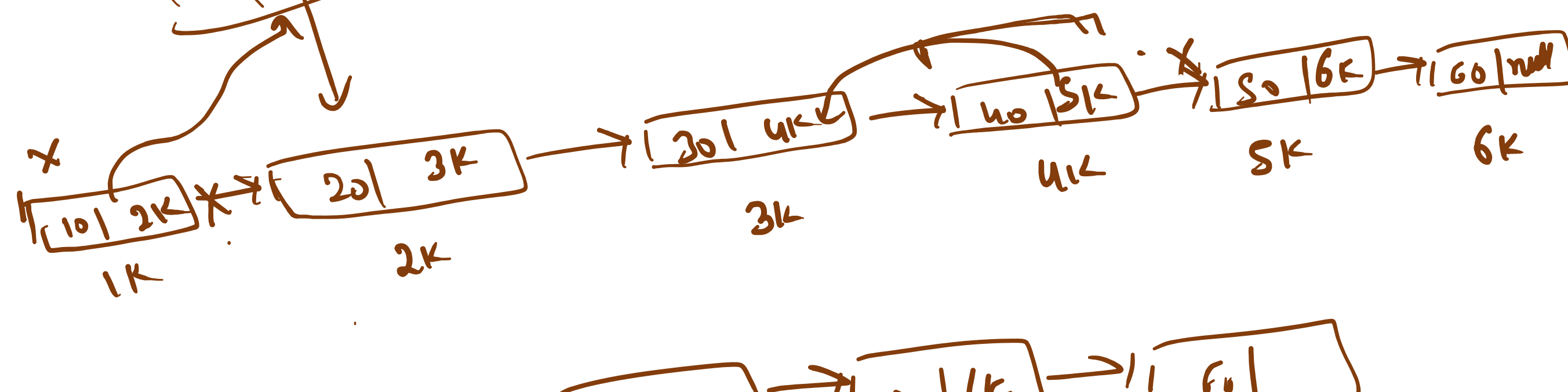
$\frac{12}{8} = 1.5$

Key
value next

Raj 87
Key-value

ll.get(index, next)

prev.next = cur.next
current = cur



Input: str = ["eat", "tea", "tan", "ate", "nat", "bat"]

x	2k	eat	tea	ate
y	3k	tan	nat	
z	6k	bat		

[eat, tea, ate]
[tan, nat]
[bat]

map.put(x).value

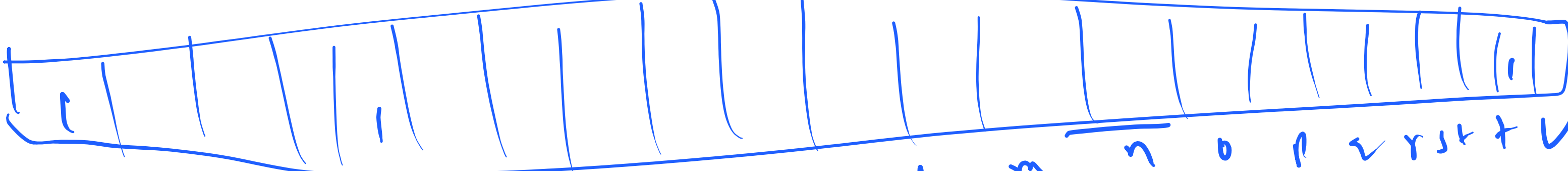
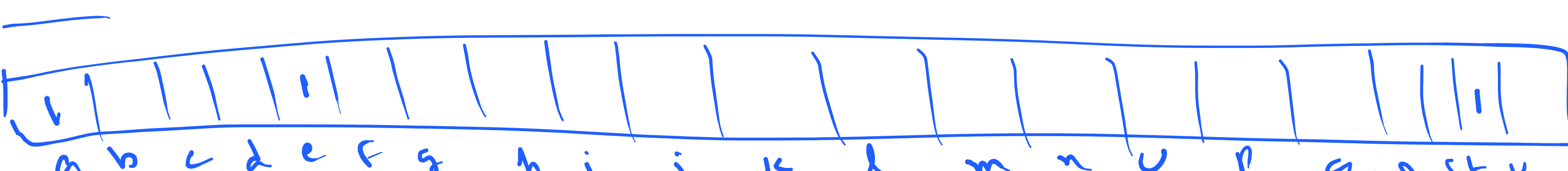
2k		eat

```
public static List<List<String>> GroupAnagram(String []arr){
    HashMap<String, List<String>> map = new HashMap<>();
    for (int i = 0; i < arr.length; i++) {
        String key = AnagramKey(arr[i]);
        if (!map.containsKey(key)) {
            map.put(key, new ArrayList<>());
        }
        map.get(key).add(arr[i]);
    }
}
```

x	2k	eat	tea	ate
y	3k	tan	nat	
z	6k	bat		

strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

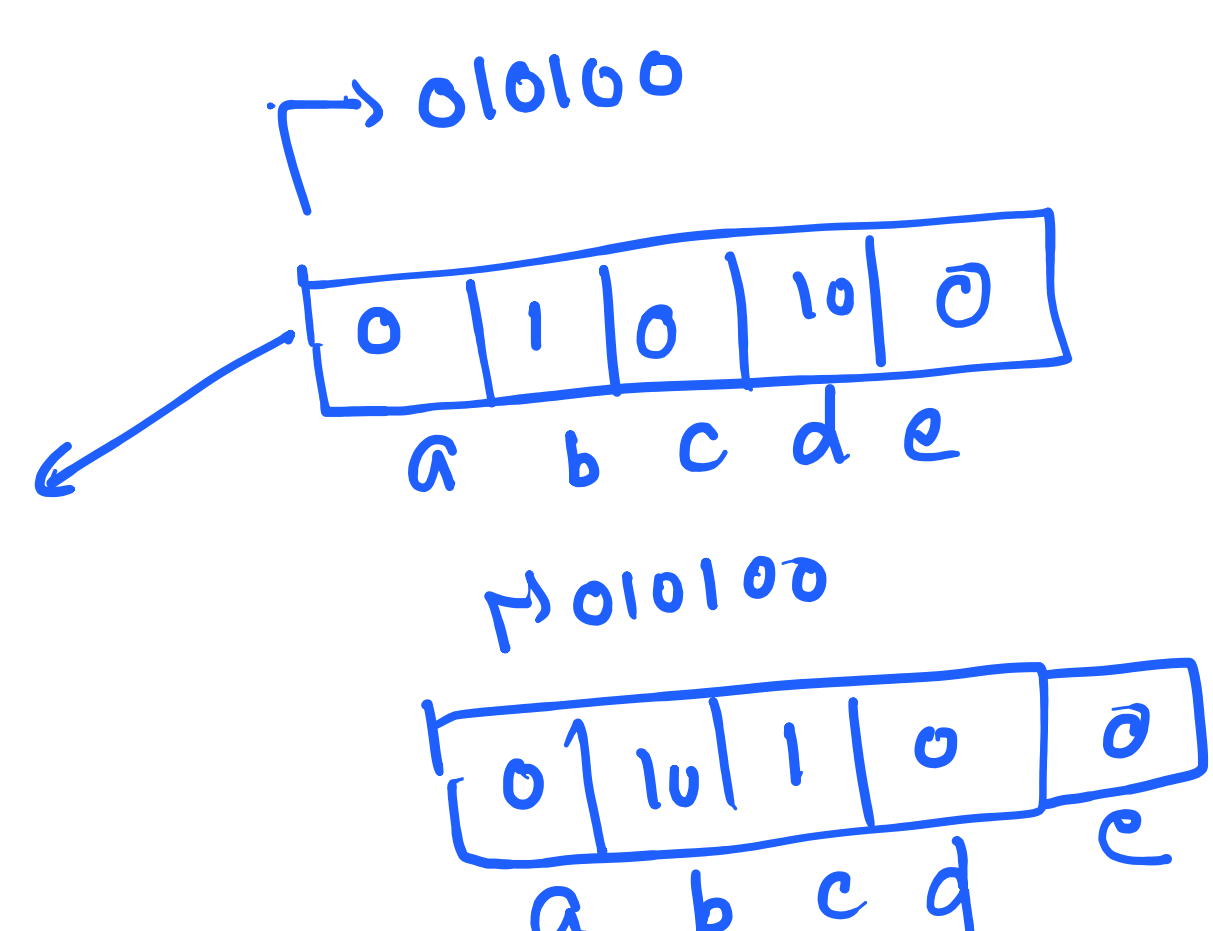
100010000000000000
100010000000000000



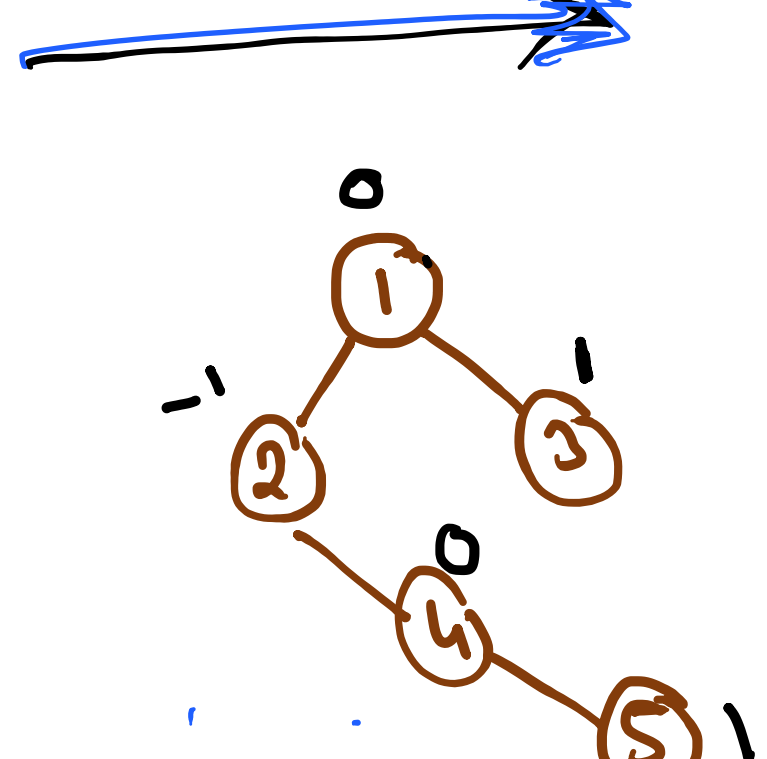
["bbbbbbbbb", "bbbbbbbbb"]

0 1 0 1 0 0

0 1 0 1 0 0



vt	data
-1	2
0	1
1	3

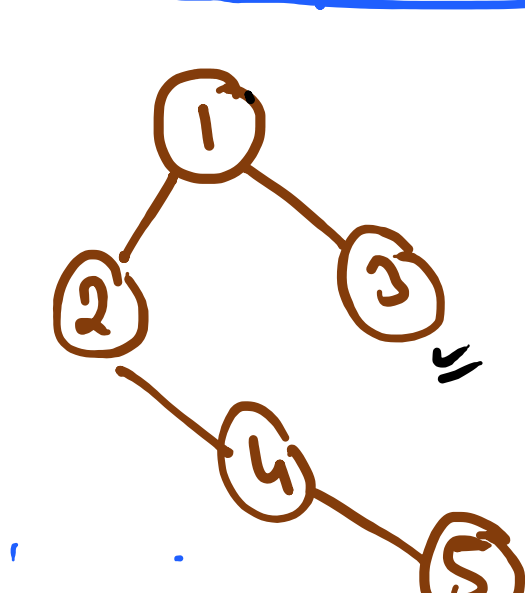


2 1 3

1/0	2/1	3/1	4/0	5/1
-----	-----	-----	-----	-----

```
public List<Integer> TopView(TreeNode root) {
    Queue<Pair> q = new LinkedList<>();
    TreeMap<Integer, Integer> map = new TreeMap<>();
    q.add(new Pair(root, 0));
    while (!q.isEmpty()) {
        Pair rp = q.poll();
        if (!map.containsKey(rp.vt)) {
            map.put(rp.vt, rp.root.val);
        }
        if (rp.root.left != null) {
            q.add(new Pair(rp.root.left, rp.vt - 1));
        }
        if (rp.root.right != null) {
            q.add(new Pair(rp.root.right, rp.vt + 1));
        }
    }
    List<Integer> ll = new ArrayList<>();
    for (int key : map.keySet()) {
        ll.add(map.get(key));
    }
    return ll;
}
```

1/0	2/1	3/1	4/0	5/1
-----	-----	-----	-----	-----



-1	2
0	1
1	3